

Deadlock prevention techniques

17 April 2026 18:18

```
package ThreadSynchronization;
public class SafeLock {
    static final Object lock1 = new Object();
    static final Object lock2 = new Object();
    static class Task implements Runnable{
        @Override
        public void run() {
            synchronized(lock1){
                System.out.println(Thread.currentThread().getName()+ " locked lock1");
            }
            synchronized(lock2){
                System.out.println(Thread.currentThread().getName()+" locked lock2");
            }
        }
    }
    public static void main(String[] args) {
        Thread t1 = new Thread(new Task(), "Thread-1");
        Thread t2 = new Thread(new Task(), "Thread-2");
        t1.start();
        t2.start();
    }
}
```

Output:

```
Thread-1 locked lock1
Thread-2 locked lock1
Thread-1 locked lock2
Thread-2 locked lock2
```

→ Both threads locks lock1 → lock2.
No circular wait
Deadlock is prevented

Prevention of Deadlock using tryLock() (ReentrantLock)

public class ReentrantLock extends [Object](#) implements [Lock](#), [Serializable](#)

A reentrant mutual exclusion [Lock](#) with the same basic behavior and semantics as the implicit monitor lock accessed using synchronized methods and statements, but with extended capabilities.

A ReentrantLock is *owned* by the thread last successfully locking, but not yet unlocking it. A thread invoking lock will return, successfully acquiring the lock, when the lock is not owned by another thread. The method will return immediately if the current thread already owns the lock. This can be checked using methods [isHeldByCurrentThread\(\)](#), and [getHoldCount\(\)](#).

The constructor for this class accepts an optional *fairness* parameter. When set true, under contention, locks favor granting access to the longest-waiting thread. Otherwise this lock does not guarantee any particular access order. Programs using fair locks accessed by many threads may display lower overall throughput (i.e., are slower; often

much slower) than those using the default setting, but have smaller variances in times to obtain locks and guarantee lack of starvation. Note however, that fairness of locks does not guarantee fairness of thread scheduling. Thus, one of many threads using a fair lock may obtain it multiple times in succession while other active threads are not progressing and not currently holding the lock. Also note that the untimed `tryLock()` method does not honor the fairness setting. It will succeed if the lock is available even if other threads are waiting.

It is recommended practice to *always* immediately follow a call to lock with a try block, most typically in a before/after construction such as:

```
class X {
    private final ReentrantLock lock = new ReentrantLock();
    // ...
    public void m() {
        lock.lock(); // block until condition holds
        try {
            // ... method body
        } finally {
            lock.unlock()
        }
    }
}
```

In addition to implementing the `Lock` interface, this class defines a number of public and protected methods for inspecting the state of the lock. Some of these methods are only useful for instrumentation and monitoring.

Serialization of this class behaves in the same way as built-in locks: a deserialized lock is in the unlocked state, regardless of its state when serialized.

This lock supports a maximum of 2147483647 recursive locks by the same thread. Attempts to exceed this limit result in `Error` throws from locking methods.

Since:

1.5

From <<https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/locks/ReentrantLock.html>>

```
package ThreadSynchronization;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
public class TryLockDemo {
    static Lock l1 = new ReentrantLock();
    static Lock l2 = new ReentrantLock();
    static class Task implements Runnable {
        @Override
        public void run() {
            try {
                if (l1.tryLock()) {
                    System.out.println(Thread.currentThread().getName() +
                        " acquired lock-1");
                    Thread.sleep(1000);
                    if (l2.tryLock()){
                        System.out.println(Thread.currentThread().getName() +
                            " acquired lock-2");
                        l2.unlock();
                    }
                }
            }
        }
    }
}
```

```

        l1.unlock();
    }
} catch (InterruptedException e) {
    e.printStackTrace();
}
}
}

public static void main(String[] args) {
    new Thread(new Task(), "Thread-1").start();
    new Thread(new Task(), "Thread-2").start();
}
}

```

Output:

```

Thread-1 acquired lock-1
Thread-1 acquired lock-2

```

Avoid nested Locks

Do not place one synchronized block inside another.

Bad practice:

```

synchronized(a1){
    synchronized(a2){
        //risky
    }
}

```

Better is:

```

synchronized(a1){}
synchronized(a2){}

```

Reduces deadlock probability.

Use Single lock(Coarse-Grained Locking)

Use one lock for all shared resources.

```

package ThreadSynchronization;
public class SingleLock {
    static final Object LOCK = new Object();
    public static void main(String[] args) {
        Thread t1 = new Thread(
            () -> {
                synchronized(LOCK){
                    System.out.println("Thread 1 is working");
                }
            }
        );
        Thread t2 = new Thread(
            () -> {

```

```

        synchronized(LOCK){
            System.out.println("Thread 2 is working");
        }
    }
);
t1.start();
t2.start();
}
}

```

Output:

```

Thread 1 is working
Thread 2 is working

```

No circular wait
Less performance

DeadLock Avoidance(Banker's Algorithm Concept)

Concept: Check system state before allocating resources.

Mostly used in operating system theory, rarely implemented in Java directly.

Thread Interruption

Interrupt long-waiting threads.
thread.interrupt()

Releases blocked Threads.
Used in controlled shutdown.

Use Concurrent Utilities

Class	Benefit
Semaphore	Controlled access
ExecutorService	Manage Threads
ConcurrentHashMap	Thread-safe
ReadWriteLock	Better control